

SafePath Transit: Crime Minimized first and last mile navigator

Woojin LIM

Simon Fraser University

Prof. gg Baker

Github: <https://github.com/RYLwoojin/SafePath-Transit-Crime-Minimized-first-and-last-mile-navigator->

1. Introduction & Problem Definition

1.1 Background: The first and last mile problem in public transit

The "First and Last mile" (FLM) in public transit refers to the gap between a traveler's starting point or final destination and a transit station. The problem I saw was that the FLM has safety issues. Because existing maps and navigation are only considering the least distance or time, not the safety. So they suggest walking in high-crime sectors. When the LFM becomes dangerous, it deter public transit use.

1.2 Problem Refinement & Client Scenario

Our problem is simple. Suggest safest FLM route to transit users by making a multi-modal navigation algorithm to minimize the exposure to the danger.

Our client is TransLink, the transit company runs business in metro Vancouver. They think Vancouver, Richmond, Burnaby and Surrey are not safe places. So, they are interested in the safety of customers. So they asked me to analyze the crime zones and make an algorithm which suggests the safest LFM route.

2. Data Acquisition, Cleaning & Integration

2.1 Heterogeneous Crime Data Integration.

A major technical hurdle involved harmonizing disparate open crime datasets from five independently governed municipalities: Vancouver (VPD), Burnaby, Surrey, Richmond, and Coquitlam. Because these datasets were distinct, they possessed heterogeneous schemas, varying coordinate reference systems (CRS), and inconsistent crime categorization taxonomies. To construct a unified spatial dataframe, a rigorous data integration pipeline was engineered:

- Coordinate Reference System (CRS) Alignment: The raw spatial data arrived in varying formats, including standard latitude/longitude coordinates (EPSG:4326) and local projected systems. To enable highly accurate, meter-based distance calculations during spatial joins, all geographic features were uniformly reprojected to the UTM Zone 10N projection (EPSG:26910). Records containing invalid or null geometries were algorithmically filtered out.
- Schema Standardization: Distinct column names across the municipal datasets, such as TYPE in Vancouver, File_Type in Burnaby, and INCIDENT_TYPE in Surrey, were standardized into a single Original_Type attribute. Temporal fields, varying from Unix timestamps to split integer columns, were parsed and consolidated into uniform YEAR and MONTH attributes.
- Categorical Harmonization: The municipalities employed different nomenclatures for identical crimes (e.g., "Break and Enter Commercial" vs. "BREAK & ENTER-BUSINESS"). A comprehensive mapping dictionary was utilized to translate these disparate local labels into generalized classifications, such as "Commercial B&E" and "Theft From Auto."

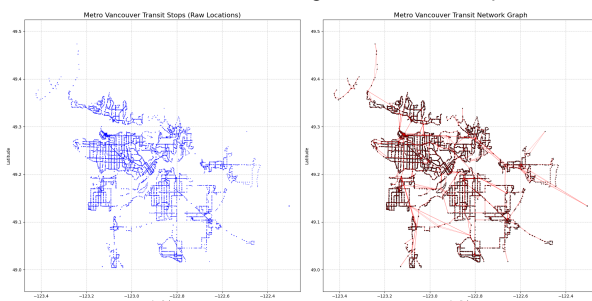
2.2 GTFS Transit Data Cleaning & Night Bus Filtering

I got the whole transit data from TransLink. They offer GTFS data which includes the bus routes info, and stops info. I excluded night buses because they only operate at limited times and cover long routes. Because of these long routes, the algorithm kept selecting them to avoid the transfer penalty, so I removed them from the transit list using pandas and `r'^N\d+'`.

2.3 Transit Transfer Graph Construction

A transit graph was constructed using the connectivity information of consecutive stops from stop_time. A weight of 10 was assigned to same-stop transfers and walking transfers.

Incorporate transfer inconvenience and time delay penalties into the algorithm, rather than using simple Euclidean distance, to avoid redundant transfers and generate realistic optimal routes.

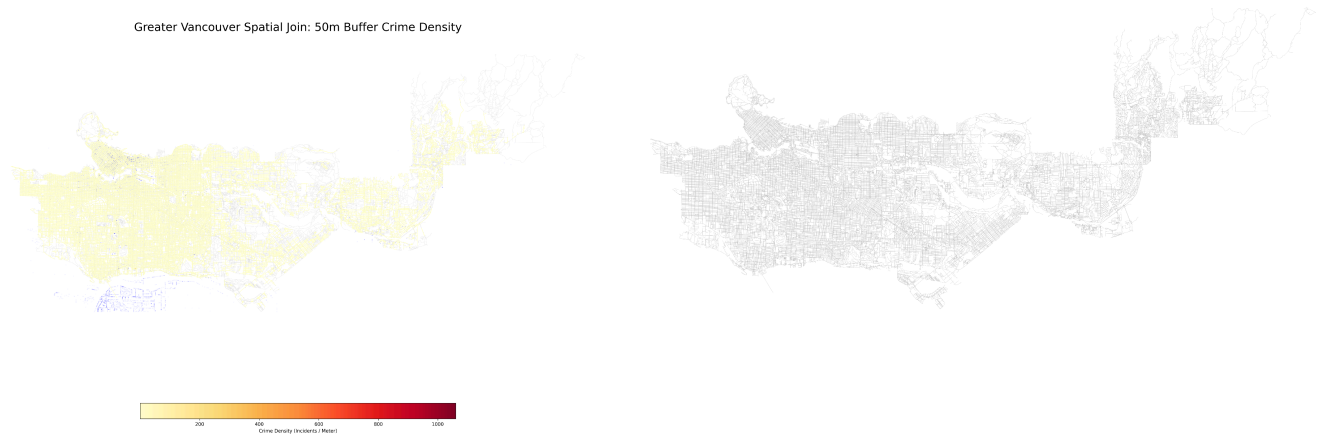


2.4 Road Network Extraction & Spatial Join with Crime Data

Using OSMnx to extract the Greater Vancouver's road network, apply a 50m buffer to road edges, and run a spatial join(`gpd.sjoin`) with crime point data to compute `crime_density` per unit length. Computing crime density normalized by road length avoids statistical distortion between long and short roads, allowing for an objective assessment of crime risk.

3. Methodology & Algorithm Design

Walkable Road Network: Greater Vancouver Region



3.1 Dynamic Local Graph Bounding for Computational Efficiency

Instead of processing the complete Burnaby road network, a local subgraph (G_{local}) is dynamically constructed by cropping nodes and edges within a 1.5 km bounding box around the origin and destination. This minimizes memory consumption during graph computations and maximizes shortest-path search performance, achieving the computational efficiency required for real-time application.

3.2 Edge Weight Normalization (Min-Max Scaling)

Without adjustment, raw distance values would dominate the cost function, making the safety weight ineffective. Normalize road length (m) and crime density (crimes/m) to a 0-1 range via Min-Max Scaling. Avoid feature dominance caused by unit differences and balance feature weights during optimal path computation in Euclidean space.

3.3 Multi-Objective Cost Function

- Shortest Cost = Normalized(Length)
- Safest Cost = $0.3 \cdot \text{Normalized(Length)} + 0.7 \cdot \text{Normalized(CrimeDensity)}$

Implement multi-objective optimization to clearly separate distance-first and safety-first routes, adjusting α and β weights to assign a 70% preference to safety

Trade-off Chart Explanation

- X-axis(Safety Weight, β): A weighting factor indicating the relative importance assigned to 'safety' during pathfinding. (0.0 considers distance only, while 1.0 considers safety only)

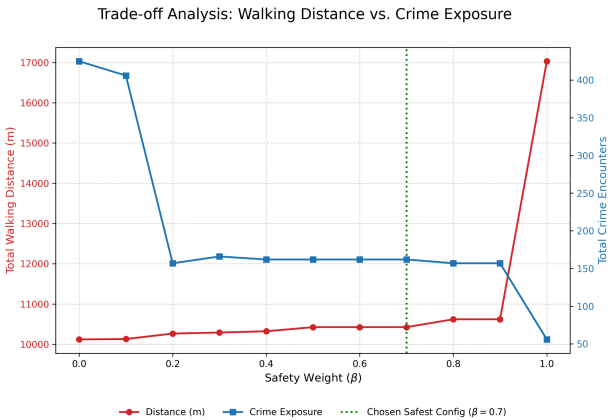
-Left Y-axis(Red): Total walking distance (in meter)

-Right Y-axis(Blue): Number of crime exposures along the route

a. $\beta = 0.0 \sim 0.2$: It gives the shortest path but the crime exposure is exceptionally high at over 400 incidents. However, increasing the weight to just $\beta=0.2$ causes a steep decline in crime exposure to approximately 150 incidents, while the walking distance increases only marginally.

b. $\beta = 1.0$: As β transitions from 0.9 to 1.0, the walking distance spikes abnormally from around 10,000m to over 17,000m. Although crime exposure reaches its minimum (approximately 50 incidents), this indicates that the algorithm forces an extreme detour that is unfeasible in real-world scenarios to avoid crime completely.

c. $\beta = 0.7$: The green vertical dashed line at $\beta = 0.7$ represents our configured algorithm setting.



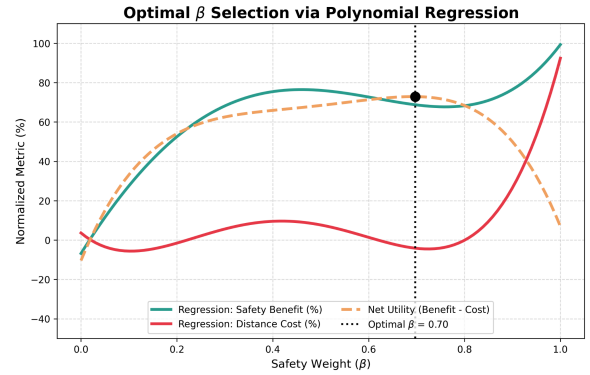
Determining the optimal β

To find the ideal β , we applied polynomial regression to the safety benefit and distance cost. By calculating the net utility (Benefit minus Cost), we identified $\beta = 0.7$ as the optimal value that maximizes the trade-off.

3.4 Integrated Multi-Modal Routing Pipeline

Evaluate direct walk condition (≤ 1.2 km) → Search transit routes using GTFS → Apply Dijkstra algorithm to first- and last-mile segments → Integrate total route

Build a flexible, complete multimodal navigation system capable of serving both short-distance walk queries and long-distance transit journeys



4. Result & Comparative Analysis

4.1 Case Study: Metropolis at Metrotown to Kentwood St.

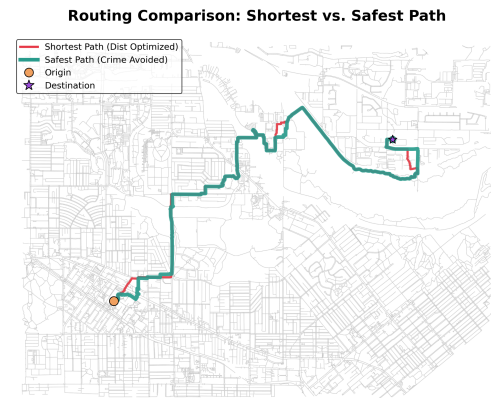
To evaluate the real-world effectiveness of the proposed multi-objective routing algorithm, a case study was conducted for a journey originating from Metropolis at Metrotown and concluding at 7710 Kentwood St. (This address was randomly chosen near SFU Burnaby).

As illustrated in the figure, the algorithm calculates two distinct paths. The Shortest path (red line) strictly minimizes physical walking distance. The Safest Path (green line, $\beta = 0.7$) applies a penalty to street segments with high historical crime densities.

As shown on the map, the Safest Path ($\beta = 0.7$, green line) diverges from the Shortest Path (red line) to bypass specific high-risk blocks while sharing major arterial roads for basic efficiency. This spatial divergence illustrates how the algorithm actively avoids localized crime hotspots. The quantitative results clearly reflect this trade-off:

- Distance Increase: Total walking distance increases by only 3.00% (from 10,120.4 m to 10,420 m).
- Crime Reduction: Spatial crime exposure drops by 61.88% (from 425 to 162 encounters).

These findings support the project's main premise: pedestrians can significantly lower their crime risk with a minimal increase in walking distance.



4.2 Geospatial Route Visualization and Mapping

To present routing results, a dual-visualization pipeline was implemented using both interactive and static mapping tools. The underlying spatial processing relies on Shapely and GeoPandas. User-input addresses were geocoded and converted into Shapely Point geometries, which were then transformed to EPSG:26910 for precise spatial alignment with the road network. For the user interface, an interactive map was built using Folium (Leaflet.js) to display multimodal trips. Walking segments for the Shortest route are rendered in red and the Safest route in green, while GTFS transit segments are drawn in blue following actual route geometries (Shapes.txt). To prevent browser lag from rendering large crime datasets and to resolve Jupyter Notebook rendering issues, the map dynamically crops crime incidents within a localized bounding box and renders directly as an HTML string.

For analytical documentation, high-resolution static plots were generated using OSMnx and Matplotlib.

4.3 Impact of Feature Normalization on Multi-Objective Optimization

A key challenge in routing was the scale difference between road length (measured in meters) and crime density (incidents per meter). Without normalization, raw distance values would dominate the cost function, rendering the safety weight (β) ineffective. To resolve this, Min-Max Scaling was applied to normalize both features into a $[0, 1]$ range within each local subgraph:

$$\text{Normalized}(\text{Length}) = \frac{\text{Length} - \text{Min}(\text{Length})}{\text{Max}(\text{Length}) - \text{Min}(\text{Length})}$$

$$\text{Normalized}(\text{Crime}) = \frac{\text{Crime Density} - \text{Min}(\text{Crime Density})}{\text{Max}(\text{Crime Density}) - \text{Min}(\text{Crime Density})}$$

This standardization ensures that both objectives contribute equally to the cost function:

$$\text{Cost}_{\text{safest}} = (0.3 \cdot \text{Normalized}(\text{Length})) + (0.7 \cdot \text{Normalized}(\text{Crime}))$$

As a result, the algorithm applies the intended weights (30% distance, 70% safety) without scale distortion, allowing Dijkstra's algorithm to correctly find the balanced path.

5. Limitation & Retrospective Discussion

5.1 Technical & Data Limitation

The current implementation has several limitations:

Temporal Unawareness: Crime data is aggregated into a static crime_density metric without temporal breakdown. Consequently, the algorithm recommends the same route at 2:00 PM and 2:00 AM, failing to capture day-night shifts in crime patterns.

Uniform Crime Weighting: All offenses are treated with equal weight in the cost function. Property crimes (e.g., bicycle theft) and violent crimes (e.g., assault) both contribute equally, which does not accurately reflect actual physical risk.

Transfer Penalty Simplification: Transfers are handled using a static penalty (weight=10) between matching stop names, without accounting for indoor walking paths, physical layouts, or real-world waiting times.

5.2 Future Work

Future iterations can address these constraints through the following enhancements:

Dynamic Temporal Routing: Incorporate the timestamp attributes (HOUR, MONTH) from the crime datasets. Filtering crime data by travel time would enable context-aware routing for day or night.

Crime Severity Index (CSI): Implement a weighted cost matrix. Assigning higher penalties to violent crimes compared to minor property offenses would yield a more realistic safety score.

GTFS-Realtime Integration: Move beyond static GTFS schedules to integrate live data feeds. Synchronizing pedestrian arrivals with real-time bus departures would reduce idle waiting times at transit stops.

6. Conclusion

This project demonstrates a data-driven approach to addressing first- and last-mile safety challenges in public transit. By combining OpenStreetMap road networks, GTFS schedules, and historical crime data, we developed a multimodal routing algorithm that balances pedestrian safety and travel efficiency.

Key geospatial techniques, such as spatial bounding, buffering, and Min-Max normalization, resolved the scale disparity between road length and crime density, allowing the multi-objective cost function to operate as intended. Case study results show that applying a balanced safety weight ($\beta = 0.7$) reduced crime exposure by 61.88% with only a 3.00% increase in walking distance. Overall, this study illustrates that incorporating localized crime data into routing algorithms provides a practical trade-off between travel distance and risk reduction, offering a framework for safer and more flexible urban navigation.

Sources:

<https://geodash.vpd.ca/opendata/#>
<https://www.translink.ca/about-us/doing-business-with-translink/app-developer-resources/gtfs>
https://opendata-surrey.hub.arcgis.com/datasets/93fc3fafda7e47c68c8dfe350c67b4e9_0/explore
<https://gis.burnaby.ca/propertycrime/>
<https://csgeo.richmond.ca/#about>

Project Experience Summary

- Developed a multimodal pedestrian routing algorithm that minimizes crime exposure by integrating GTFS transit data, OpenStreetMap networks, and municipal crime datasets using GeoPandas, NetworkX, and OSMnx.
- Implemented a multi-objective cost function that combines localized crime densities and Min-Max scaled distance features, achieving a 61% reduction in crime exposure with a 3% increase in walking distance.
- Built an interactive Leaflet.js-based mapping pipeline using Folium to visualize routing results, incorporating dynamic data filtering and HTML conversion to optimize rendering performance.